

FDMJ 完整编译器技术报告

代码包根目录：`Compiler/`

主编译器入口：`tools/main/main.cc` → `build/tools/main/fdmjc`

流水线调度：`lib/pipeline/pipeline.cc`

本报告对应 README：`README.md`

目录

1. 引言
2. 词法分析与语法分析
3. 类型检查（语义分析）
4. 翻译为 IR+
5. Quad 表示的形成
6. 静态单赋值形式（SSA）
7. SSA Quad 优化
8. ARM 指令选择
9. 寄存器分配
10. ARM 汇编代码生成
11. 结论与用户手册

一、引言

1.1 报告主题与目标读者

本报告描述了一个完整的 **FDMJ 完整编译器** 的实现方式。目标读者是具备基础编译原理知识、但尚未修读本课程的计算机科学学生。报告重点说明本仓库中的代码如何组织、如何协同工作，并在每一阶段用同一个示例程序 `quadtest2` 展示数据如何变换。

阅读本报告时，建议在 IDE 中打开同名源文件；各节标题与 `lib/`、`include/` 下的模块目录一一对应。

1.2 整体编译流程

本编译器将 FDMJ 源程序（`.fmj`）经以下阶段变为可在 QEMU 上运行的 ARM 汇编（`.colored.s`）：

```
.fmj
→ [Lex/Yacc] 抽象语法树 AST
→ [语义分析] 带类型信息的 AST (XML)
→ [ast2tree] IR+ 树 (XML)
→ [canon] 规范化 IR+
→ [tree2quad] Quad (含 def/use)
→ [blocking + 数据流] 基本块与控制/数据流
→ [quad2ssa] SSA Quad (含 φ)
→ [opt / loopHoist / loopIV] 优化后的 SSA Quad
→ [指令选择 + 调度] 未着色 ARM 汇编
```

```
→ [buildIg + coloring] 寄存器分配
→ [asmprog2colored] 最终 .colored.s
```

流水线由 `compiler::compileProgram ( lib/pipeline/pipeline.cc )` 统一驱动。`fdmjc` 对每种优化模式 (`base`、`const`、`loop1`、`loop2`、`allloop`、`allopt`) 各生成一套中间文件与 `.colored.s`。

1.3 代码目录结构

目录	职责	报告章节
<code>lib/frontend/</code>	Flex 词法 (<code>lexer.ll</code>)、Bison 语法 (<code>parser.yy</code>)	§2
<code>lib/ast/</code>	语义分析、符号表 (<code>Name_Maps</code>)	§3
<code>lib/ir/</code>	AST→IR+ (<code>ast2tree.cc</code>)、规范化 (<code>canon.cc</code>)	§4
<code>lib/quad/</code>	IR+→Quad (<code>tree2quad.cc</code>)、分块 (<code>blocking.cc</code>)、SSA (<code>quadssa.cc</code>)	§5–§6
<code>lib/opt/</code>	常量传播、循环外提、归纳变量与强度削弱	§7
<code>lib/instr/</code>	指令选择、调度、汇编程序表示	§8
<code>lib/reg/</code>	干扰图构建、图着色寄存器分配	§9–§10
<code>lib/pipeline/</code>	端到端编排、输出文件命名	全文
<code>lib/util/</code>	AST/IR/Quad/Flow/Asm 的 XML 读写	辅助
<code>test/</code>	46 个 <code>.fmj</code> 测试源文件	§11
<code>vendor/libsysy/</code>	SysY 运行时 (<code>ARM</code> , <code>libsysy32.s</code>)	§11

1.4 贯穿示例：`quadtest2`

本报告以 `test/quadtest2.fmj` 为主线。该程序包含：

- `main` 方法与 `public class fib`；
- 局部变量、`new fib()`、虚方法调用 `o.f(i)`；
- `getint()` 输入、`putint/putch` 输出；
- `if` 条件返回、`while` 循环；
- 递归方法 `fib.f ( this 与间接调用 )`。

输入 5 时，期望输出：

```
Enter the number of term:0 1 1 2 3
```

(`make run` 通过 `scripts/test_manifest.json` 自动向程序 `stdin` 注入 `5\n`，无需手动输入。)

二、词法分析与语法分析

2.1 词法分析 (Flex)

源文件：lib/frontend/lexer.ll

生成物：构建时由 Flex 生成 build/lib/frontend/lexer.cc，封装为 ASTLexer (include/frontend/ASTLexer.hh)。

词法器识别 FDMJ 关键字 (如 public、class、while、getint)、标识符、非负整数、运算符与分隔符。单行注释 // 与块注释 /* ... */ 分别进入 COMMENT1、COMMENT2 状态并丢弃。每个 token 通过 YY_USER_ACTION 调用 copyLocation()，为后续 AST 节点记录行列号 (ast_location)。

在 quadtest2 中：例如 getint 映射为 Token::GETINT，class fib o 中的 class、fib (标识符)、o 依次产出相应 token。

2.2 语法分析 (Bison) 与 AST 构造

源文件：lib/frontend/parser.yy

入口函数：fdmjParser (文件末尾)，由 pipeline.cc 的 ensureFrontend 调用。

语法分析器使用 Bison C++ 接口 (ASTParser)，%define parse.error detailed 启用详细错误信息。每个产生式动作在 lib/ast/FDMJAST.cc 所定义的节点类型上构造 AST，例如：

- PROG → Program(main, classDeclList);
- CLASSDECL → ClassDecl;
- METHODDECL → MethodDecl;
- WHILESTM → While; IFSTM → IfElse。

示例：quadtest2 中 while (i < n) 的 AST

解析后得到 While 节点，其子节点为：

- 条件：Op(<)，左操作数为 IdExp("i")，右操作数为 IdExp("n")；
- 循环体：StmList，内含 PutInt、PutCh、Assign(i = i + 1)。

Assign 节点在语法动作中绑定源位置 Pos (来自 calcLocation)，写入 .2-semant.ast 时可精确定位到源行。

2.3 解析错误报告

当 token 序列不符合文法时，Bison 调用：

```
void ASTParser::error(const location_t &location, const std::string
&message)
{
    std::cerr << "Error at lines " << location << ": " << message <<
std::endl;
}
```

(lib/frontend/parser.yy 约第 782 行。)

`fdmjParser` 若解析失败返回 `nullptr`，流水线打印 `Parse failed` 并中止。非法字符由词法器规则，分支报告 `Illegal input`。

三、类型检查（语义分析）

3.1 总体流程

源文件：`lib/ast/semantanlyzer.cc`（访问者 `AST_Semant_Visitor`）、`lib/ast/setnamemaps.cc`（`AST_Name_Map_Visitor`）、`lib/ast/namemaps.cc`（符号表实现）。

入口：`semant_analyze(Program*)`，在 `ensureFrontend` 中于解析成功后调用。

步骤：

- 1. `makeNameMaps(node)`：第一遍遍历 AST，构建 **Name_Maps**（类、方法、形参、局部变量、继承关系）；
- 2. `AST_Semant_Visitor::visit`：第二遍进行类型检查，为每个表达式/语句填充 `SemantInfo`（类型、`s_kind`、`lvalue` 等）；
- 3. 结果写入 `AST_Semant_Map`，并通过 `saveSemantAst` 输出 XML：`test/quadtest2.2-semant.ast`。

3.2 构建的表格（Name_Maps）

`Name_Maps`（`include/ast/namemaps.hh`）是本阶段的核心数据结构，主要包括：

表	内容	quadtest2 示例
类表	类名、父类、类变量	<code>fib</code> 类
方法表	每类的方法列表	<code>fib.f</code> 、 <code>__\$main__.main</code>
方法形参表	形参名 → <code>Formal</code>	<code>fib.f</code> 的 <code>n</code> 、 <code>^return^_f</code>
方法局部变量表	变量名 → <code>VarDecl</code>	<code>main</code> 的 <code>n,i,o</code>
虚方法全局索引	方法在 <code>vtable</code> 中的位置	<code>fib^f</code> → 槽 0

语义分析还为每个变量/形参分配 **临时变量编号（temp num）**，供 IR 阶段使用。`quadtest2.fmj` 末尾注释与 `.2-semant.ast` 中一致，例如 `main` 中 `i→100`，`n→101`，`o→102`。

3.3 类型检查内容与错误报告

访问者对各类 AST 节点执行规则检查，例如：

- `If/While` 的条件必须是 `int` 表达式；
- `Assign` 左右类型必须兼容；
- 类继承不能成环、不能自继承；
- 方法调用实参与形参个数、类型匹配；
- `new fib()` 的类名必须已声明。

错误时向 `stderr` 打印 `Error: ...` 及位置（`node->getPos()->print()`），并设置 `semant_visitor.mode = -1`。`semant_analyze` 检测到 `mode == -1` 后返回 `nullptr`，编译中止。

示例：若将 `while (i < n)` 改为 `while (o)`，访问 `While` 时会报告条件必须为 `int` 表达式。

3.4 输出：带语义信息的 AST

`quadtest2.2-semant.ast` 为 XML，根节点 `<FDMJAST>` 含 `<NameMaps>` 与带 `typeKind`、`s_kind`、`lvalue` 属性的 AST 子树。后续 `ast2tree` 直接读取该文件（或通过内存中的 `AST_Semant_Map`）生成 IR。

四、翻译为 IR+

4.1 入口与类布局

源文件：`lib/ir/ast2tree.cc`
入口：`ast2tree(Program*, AST_Semant_Map*)`
输出：`test/quadtest2.3.irp`（XML 格式的 IR+ 树，由 `tree2xml` 写入）。

翻译前，`generate_class_table` 根据 `Name_Maps` 构建 `Class_table`：

- `var_pos_map`：每个类字段在统一对象记录（Unified Object Record）中的偏移；
- 全局虚函数表：所有类的方法按声明顺序编号（`fib^f` 等方法名写入 `Name` 节点）。

4.2 AST 组件到 IR+ 的映射（以 `quadtest2` 为例）

AST 构造	IR+ 构造	示例
局部变量	<code>Temp(type, num)</code>	<code>i → Temp(INT, 100)</code>
赋值	<code>Move(temp, exp)</code>	<code>i = 0 → Move(t100, Const(0))</code>
<code>new fib()</code>	<code>ESeq + ExtCall(malloc) + 向 vtable 槽写入 Name("fib^f")</code>	见 <code>.3.irp</code> 第 13–36 行
<code>o.f(i)</code>	<code>Call</code> / 嵌套在 <code>Move</code> 中	加载 <code>vtable</code> 函数指针后 <code>Call</code>
<code>getint()</code>	<code>ExtCall("getint")</code>	<code>Move(t101, ExtCall(getint))</code>
<code>if (n<0 n>47)</code> <code>return -1</code>	<code>CJump + 多个 Label + Return</code>	条件链式跳转
<code>while (i < n)</code>	<code>Label + CJump + 循环体 + Jump 回标签</code>	标准 <code>while Lowering</code>
<code>return 0</code>	<code>Return(Const(0))</code>	<code>main</code> 末尾

递归方法 `fib.f` 翻译为独立 `FunctionDeclaration name="fib^f"`，参数为 `_^this^_`（PTR）与 `n (INT)`。`this.f(n-1)` 通过从对象头加载 `vtable` 指针，再 `Call` 实现。

4.3 IR+ 规范化（Canon）

源文件：`lib/ir/canon.cc`
输出：`test/quadtest2.3-canon.irp`

`canon_visitor` 对 IR 做：

- 线性化嵌套 `Seq`；
- 消除不必要的 `ExpStm`（纯常量副作用除外）；
- 将复杂表达式拆成 **线性语句序列 + 单一结果 Temp**（ESeq 展开），便于后续“铺砖”匹配 Quad 瓦片。

规范化后的 IR 是 `tree2quad` 的直接输入。

4.4 IR 阶段使用的“表”

除 `Class_table` 外，翻译过程还维护：

- **Temp_map**：分配 fresh label/temp；
- **Method_var_table / Formal 映射**：AST 变量名 → IR Temp 编号（来自语义分析）。

这些表在 `ast2tree` 的访问者类中按当前“类^方法”上下文查询。

五、Quad 表示的形成

5.1 IR+ → Quad

源文件：`lib/quad/tree2quad.cc`

入口：`tree2quad(Program*)`

输出：`test/quadtest2.4.quad`（文本）、`test/quadtest2.4-xml.quad`（XML）

`Tree2Quad` 访问者逐函数、逐语句将 IR 节点 lowering 为 `QuadStm` 序列，并组织为 `QuadProgram` → `QuadFuncDecl` → `QuadBlock` → `quadlist`。

quadtest2 片段（main 中读输入与循环）——摘自 `quadtest2.4.quad`：

```
MOVE_EXTCALL t101:int <- getint(); def: t101 use:
CJUMP < t101:int Const:0? L105 : L104; def: use: t101
...
LABEL L110;
CJUMP < t100:int t101:int? L111 : L112; def: use: t100 t101
MOVE_CALL t133:int <- f[...] (t102:ptr, t100:int); def: t133 use: ...
MOVE_BINOP t100:int <- (+, t100:int, Const:1); def: t100 use: t100
JUMP L110;
```

5.2 基本块划分（Blocking）

源文件：`lib/quad/blocking.cc`

在 SSA 之前，`blocking` 将每个函数内的 Quad 序列按 **LABEL / JUMP / CJUMP / RETURN** 切分为基本块，并刷新 `Exit labels` 集合。这是后续控制流分析与 SSA 的前提。

5.3 瓦片（Tiles）与铺砖过程

本实现中，IR $\rightarrow$ Quad 的“铺砖”体现为 **Tree2Quad** 访问者中每个 IR 节点类型对应一组 **Quad** 模板（而非单独的 tile 文件）。典型瓦片包括：

IR 模式	Quad 指令
Move(dst, src)	QuadMove
BinOp 作为 Move 右值	QuadMoveBinop
Mem(BinOp(+,ptr,off))	QuadPtrCalc + QuadLoad/QuadStore
Call / 带返回值的调用	QuadCall / QuadMoveCall
外部函数	QuadExtCall / QuadMoveExtCall
条件跳转	QuadCjump
无条件跳转	QuadJump
返回	QuadReturn

复杂表达式在 canon 阶段已拆成多条 Move/BinOp，Tree2Quad 逐条匹配上述瓦片。

5.4 每条 Quad 的 def / use 集合

源文件：`lib/quad/tree2quad.cc` 文件开头辅助函数及各类 emit 逻辑。

规则（与本代码一致）：

- **QuadMove** : def = {dst} , use = src 中的 Temp ;
- **QuadMoveBinop** : def = {dst} , use = {left, right} 中的 Temp ;
- **QuadLoad** : def = {dst} , use = {ptr} ;
- **QuadStore** : def = $\emptyset$, use = {dst, src}（内存地址与值）；
- **QuadCall / QuadMoveCall** : def 为返回值 Temp（若有），use 为 obj 与所有实参 Temp ；
- **QuadCjump** : def = $\emptyset$, use = {cond_left, cond_right} ;
- **QuadReturn** : use = {返回值 Temp 或空} ；
- **LABEL / JUMP** : 通常 def = use = $\emptyset$ 。

生成时在构造函数中传入 `defSet(...)/useSet(...)`，并打印在 `.quad` 文件每行末尾（见 `quadtest2.4.quad` 的 `def: /use:` 后缀）。

这些集合在 `lib/quadflow/dataflowinfo.cc` 中用于活跃变量分析，也是 SSA 插入 ϕ 的基础。

六、静态单赋值形式（SSA）

6.1 实现入口

源文件：`lib/quad/quadssa.cc`

入口：`quad2ssa(set<FuncFlowInfo*>)`（由 `pipeline.cc` 的 `ensureSsa` 调用）

前置步骤（`ensureSsa`）：

1. `blocking(quad)` 分块；

- 2. `dataFlowProg + computeLiveness` 计算数据流；
- 3. `ControlFlowInfo::computeEverything` 构建控制流图；
- 4. 输出 `test/quadtest2.4-quadwithflow-xml.quad` (含流图与活跃信息)。

6.2 ϕ 函数插入算法 (本代码)

SSA 转换采用 **Cytron** 等基于支配边界 (dominance frontier) 的标准算法，实现要点：

- 1. 对每个 Temp，收集 **def 基本块** 集合；
- 2. 在 def 块的支配边界上放置 **候选 ϕ** (`QuadPhi`)；
- 3. 按 CFG 顺序遍历，维护每个逻辑 Temp 的 **版本栈**；
- 4. 重命名：每次 def 分配新版本号 `oldNum*100+ver` (见 `VersionedTemp::versionedTempNum`)；
- 5. 填充 ϕ 的参数：对每个前驱块，从栈顶取当前版本写入 (`predLabel, temp`)；
- 6. `refreshPhiDefUse` 更新 ϕ 的 def/use 集合。

quadtest2 示例：编译时在 stderr 可见诊断 (`quadssa_diag.cc`)：

```
SSA_DIAG_BEGIN func=fib^f
SSA_TEMP_DIAG temp=t100 candidate_phi=[L104,L105,L106]
SSA_DIAG_END func=fib^f
```

表示逻辑 Temp `t100` (`^this^`) 在 `fib.f` 的多个汇合块需要 ϕ 。SSA 化后，版本化 Temp 如 `t10000`、`t10100` 出现在 `quadtest2.4-ssa-opt.quad` 等文件中。

6.3 SSA 产物文件

文件	含义
<code>*.4-ssa-xml.quad</code>	SSA Quad 的 XML
<code>*.4-ssa-withflow-xml.quad</code>	SSA + 控制/数据流 (后续优化与 codegen 输入)
<code>*.4-ssa.quad</code>	SSA 文本 listing (由 <code>writeQuadText</code> 生成；部分构建下可能为空，以 XML 为准)
<code>*.reflow-<opt>.4-ssa-withflow-xml.quad</code>	各优化 pass 后 reflow 的流图

七、SSA Quad 优化

优化在 `lib/pipeline/pipeline.cc` 的 `buildOptimizedWithflow` 中按模式组合调用。

7.1 Constant Propagation (常量传播 / SCCP)

源文件：`lib/opt/opt.cc`

函数：`optProg(QuadProgram*)`

输出示例：`test/quadtest2.4-ssa-opt.quad`

算法在 SSA 上使用 **格值 (Lattice)** : `NO_VALUE` $\rightarrow$ `ONE_VALUE(k)` $\rightarrow$ `MANY_VALUES` , 对每条 Quad 做 meet-over-paths 传播。可常量折叠的 `MOVE`、`MOVE_BINOP` 会被替换为常量或删除不可达块。

quadtest2 上的效果 : main 入口对 `t102`、`t100` 的初始 `Move` 与大量 `putch` 的常量参数在 `const` 模式下被进一步简化 ; `fib.f` 中不可达的 `φ` 参数被清理。

7.2 Loop Invariant Hoisting (循环不变式外提 , loop1)

源文件 : `lib/opt/loophoistfunc.cc`、`lib/opt/findloopheader.cc`
函数 : `findLoopHeaders` + `loopHoistFunc`

步骤概要 :

- 1. 基于回边识别循环头 (loop header) ;
- 2. 对每个循环 , 找出 在所有迭代中值不变的 Quad (使用 def-use 与循环内定义信息) ;
- 3. 将此类 Quad 提升到循环头之前的新基本块 ;
- 4. 通过 `reflowFromProgram` 重建流图。

quadtest2 : `while ( i < n )` 循环体中对象指针 `t102`、`vtable` 加载等在 `loop1` 模式下可部分外提。更明显的案例见 `test/optloopivtest1.fmj` (循环内 `4*k+2` 的计算)。

7.3 Loop Induction Variables & Strength Reduction (loop2)

源文件 :

- `lib/opt/loopinductionbasic.cc`、`loopinductionderived.cc` — 识别基本/派生归纳变量 ;
- `lib/opt/loopstrengthreduction.cc` — 强度削弱 ;
- `lib/opt/loopinductionelimination.cc` — 消除冗余归纳变量 ;
- 入口 : `loopInductionStrengthReductionPass`、`loopInductionCleanupPass`

对 `optloopivtest1.fmj` :

```
while (k>0) {
    k = k-1;
    putint(4*k+2);
}
```

`loop2` 将 `4*k+2` 转为对派生归纳变量的增量更新 , 减少每次迭代的乘法。输出见 `test/optloopivtest1.loop2.colored.s` 与 `*.4-ssa-loopivopt.quad`。

7.4 优化模式组合

模式	启用的 Pass
base	无 (仅 SSA)
const	optProg
loop1	loopHoistFunc
loop2	归纳变量 + 强度削弱

模式	启用的 Pass
<code>allloop</code>	<code>loop1 + loop2</code>
<code>allopt</code>	<code>const + loop1 + loop2</code>

每次优化后调用 `reflowFromProgram ( lib/pipeline/reflow.cc )` 重新 blocking、计算活跃性并写回流图 XML。

八、ARM 指令选择

8.1 流水线位置

源文件：`lib/instr/selectInstr.cc` (核心)、`lib/instr/buildAdvDFG.cc`、`lib/instr/buildPreSchedule.cc`

调用链 (`pipeline.cc` → `codegenFromWithflow`) :

1. `buildAdvDFGprog` — 基于 SSA Quad 构建高级 DFG ;
2. `buildPreScheduleProg` — 准备调度输入 ;
3. `runInstructionSelectionPass` — Quad → `AssemInstr` 列表 ;
4. `scheduleProg` — 指令调度与函数级 prologue/epilogue ;
5. `schedule2xml` / 读取 bootstrap → `*.5-xml.asm`。

8.2 函数入口、调用与返回 (以 `quadtest2` 为例)

函数入口 (`lib/instr/schedule.cc`) : 每个函数开头插入 ARM prologue , 例如 `main` 中 :

```
push {r4-r10, fp, lr}
sub sp, sp, #4
add fp, sp, #36
```

外部调用 (SysY 运行时) : `QuadExtCall` / `QuadMoveExtCall` 选择为 `bl getint`、`bl putint`、`bl putch`、`bl malloc` 等 (`selectInstr.cc` 中 `appendOper(bundle, "bl " + extfun, ...)`)。参数按 ARM 调用约定放入 `r0-r3`。

方法调用 : `QuadMoveCall` 加载 vtable 函数地址后 `bl fib$f` (名称由类名与方法名 mangling 得到)。

返回 : `QuadReturn` 生成 `mov r0, ... + epilogue` (恢复 `fp/sp/lr`) + `bx lr`。main 返回 0 时可见 `mov r0, #0` 与栈恢复序列。

条件分支 : `QuadCjump lowering` 为 ARM 条件码比较与 `beq/bne/blt` 等, 基本块标签映射为 `main$L110` 等形式。

8.3 指令选择策略

选择器按 Quad 种类分派, 使用 `StatementBundle` 收集一个 Quad 对应的多条 ARM 伪指令 (`AssemInstr::Move/Oper`)。常量 materialization 使用 `movw/movt` 或适合 8 位旋转 immediate 的 `mov` (见 `fitsMovImmediate`)。

输出未着色汇编：`test/quadtest2.base.s`、`test/quadtest2.base.5-xml.asm`。

九、寄存器分配

9.1 干扰图构建

源文件：`lib/reg/buildIg.cc`、`lib/instr/asmdataflow.cc`

- 1. `preDataFlowPass(asmProg)` 在汇编指令级做活跃变量分析；
- 2. `buildIg` 对每个函数，自底向上扫描指令：在每条定义点，让 **live-out** 中与 **def** 不同的 **Temp** 与 **def** 互相连边；
- 3. 识别 `mov` 形式的 **move 对**，供保守合并（`coalesce`）使用。

9.2 图着色算法

源文件：`lib/reg/coloring.cc`

入口：`coloring(InterferenceGraph*, k)`，`pipeline.cc` 中 **k = 9**（可使用 `r4-r10` 等 callee-saved 与 scratch）。

实现 **Chaitin-Briggs 风格** 的 `simplify/select`：

- 简化度 < k 的节点入栈；
- 处理 `move` 合并；
- 着色失败则 `spill` 标记；
- 最终得到 `colors: TempNum → 物理寄存器编号` 及 `coalescedMoves`、`spilled` 集合。

9.3 着色结果应用

源文件：`lib/instr/asmprog2colored.cc`

函数：`asmprog2colored(AsmProg*, vector<Coloring*>)`

将每条 `AssemInstr` 中的 `Temp` 替换为 `r4-r10` 等物理寄存器名；对 `spill Temp` 生成额外的 `ldr/str` 访问栈槽。最终字符串写入 `test/quadtest2.base.colored.s`。

十、ARM 汇编代码生成

10.1 最终文件

文件	说明
<code>*.<opt>.s</code>	调度后的汇编（未着色或中间形式）
<code>*.<opt>.5-xml.asm</code>	汇编的 XML 表示
<code>*.<opt>.colored.s</code>	寄存器分配后、可链接的最终 ARM 汇编

`quadtest2.base.colored.s` 片段：

```
.global main
main:
main$L113:
    push {r4-r10, fp, lr}
    ...
    bl getint
    mov r5, r0        @ n 在 r5
    ...
    bl fib$f
    bl putint
```

10.2 与 SysY 运行时链接

Makefile 中：

```
GCC = arm-linux-gnueabi-gcc ... -static
LIBSYSY = vendor/libsysy/libsysy32.s
```

`scripts/run_tests.py` 将 `*.colored.s` 与 `libsysy32.s` 链接为 ARM 可执行文件，再用 `qemu-arm` 运行。

十一、 结论与用户手册

11.1 总结

本编译器在 `Compiler/` 目录下整合了 FDMJ 前端（Flex/Bison + 语义分析）、IR+ 与 Quad 中间表示、SSA、三类 SSA 优化、ARM 指令选择、图着色寄存器分配及汇编生成，通过 `fdmjc` 与 Makefile 目标对外提供统一接口。报告各节所述机制均可在对应源文件中找到实现；贯穿示例 `quadtest2` 展示了从含类与递归的源程序到可运行 ARM 代码的完整路径。

11.2 环境要求

- 64 位 Ubuntu 20.04+ (x86)
- 工具：`cmake`、`ninja`、`flex`、`bison`、`g++`、`make`
- 运行：`arm-linux-gnueabi-gcc`、`qemu-arm`

11.3 构建与测试

```
cd Compiler
make build      # 构建 fdmjc
make compile    # 编译 test/*.fmj，生成全部中间文件与 6 种 .colored.s
make run        # QEMU 运行 base（无优化）– 批量测试，stdin 来自
test_manifest.json
make run-const  # 常量传播
make run-loop1  # 循环不变式外提
make run-loop2  # 归纳变量与强度削弱
```

```
make run-allloop
make run-allopt # 全部优化
make clean      # 清理生成物
```

验证脚本（可选）：

```
make verify          # 结构 + IR + 快速端到端
make verify-e2e-full # 完整 manifest 用例
```

11.4 单程序编译

```
./build/tools/main/fdmjc test quadtest2 all      # 六种优化级别
./build/tools/main/fdmjc test quadtest2 base    # 仅 base
./build/tools/main/fdmjc test quadtest3 quad-only # 仅到 Quad
```

11.5 主要输出文件（每个 <name>.fmj）

后缀	阶段
.src	去注释源 listing
.2-semant.ast	语义 AST (XML)
.3.irp / .3-canon.irp	IR+ / 规范化 IR+
.4.quad / .4-xml.quad	Quad
.4-quadwithflow-xml.quad	Quad + 流图（SSA 前）
.4-ssa-xml.quad / .4-ssa-withflow-xml.quad	SSA Quad + 流图
.4-ssa-opt.quad 等	各优化级别 Quad
.<opt>.s / .<opt>.5-xml.asm	汇编（指令选择/调度）
.<opt>.colored.s	寄存器分配后的 ARM（QEMU 运行）

11.6 手动交互运行（需自行输入时）

```
arm-linux-gnueabi-hf-gcc -mcpu=cortex-a72 -Wall -static -Wl,-z,noexecstack
-lm \
  -o /tmp/t test/quadtest2.base.colored.s vendor/libsysy/libsysy32.s
qemu-arm /tmp/t
# 输入 5 并回车
```

11.7 测试集

test/ 含 46 个 .fmj : IR (irtest1–22)、Quad (quadtest1–9、simplecall1/2)、后端 (insttest0–4、fibonacci、bubblesort、optloopivtest1–6)。用例由 scripts/setup_test.sh 从课程 HW 目录安装。

11.8 实现说明 (与课程代码的关系)

- **前端 parser/lexer** : 基于课程 PARSING 提供的 Flex/Bison 框架 , AST 节点定义于 include/ast/FDMJAST.hh。
- **后端 bootstrap 测试** (insttest*、fibonacci、bubblesort、optloopivtest*) : 若已存在预置的 *.4-ssa-withflow-xml.quad (来自 HW11/HW12) , compileProgram 会 **跳过** 从 .fmj 重新生成 AST/IR/Quad/SSA , 直接从 SSA 流图进入优化与 codegen ; 完整前端路径请以 quadtest*、irtest* 等用例为准。
- **报告与代码交叉引用** : 建议在阅读某模块时打开本报告对应章节 ; 关键入口函数均已在各节标明路径。